Análisis y Diseño de Algoritmos Avanzados

Departamento de Ciencias de la Computación

Nombre del Estudiante: _____

Matrícula: _____ **Fecha:** _____

Ejercicio 1: Assign Cookies (25 pts)

Dadas dos listas de enteros `g` y `s`, donde `g[i]` representa el tamaño mínimo de galleta que satisface al niño i y `s[j]` representa el tamaño de la galleta j, determine el máximo número de niños que pueden quedar satisfechos. (Indique la complejidad temporal y espacial de su algoritmo).

Cada niño puede recibir como máximo una galleta y cada galleta puede asignarse a lo más a un niño.

Ejemplo:

```
g = [1,2,3],    s = [1,1]
```

Salida:

Ejercicio 2: Pow(x, n) (22.5 pts)

Dados un número real `x` y un entero `n`, calcule el valor de x^n sin utilizar el operador de exponenciación de Python. (Indique la complejidad temporal y espacial de su algoritmo).

Ejemplo:

`x = 2.0,  n = 10`

Salida:

1024,0

Ejercicio 3: Search in Rotated Sorted Array (25 pts)

Dado un arreglo `nums` de enteros distintos, originalmente ordenado de forma ascendente y posteriormente rotado, y un entero `target`, encuentre el índice donde aparece `target`.

Si el elemento no existe, regrese `-1`. (Indique la complejidad temporal y espacial de su algoritmo).

Ejemplo:

`nums = [4,5,6,7,0,1,2],` `target = 0`

Salida:

4

Ejercicio 4: N-Knights (25 pts)

Dado un entero positivo `n`, coloque exactamente `n` caballos en un tablero de ajedrez de tamaño $n \times n$ de manera que ningún caballo pueda atacar a otro.

Regrese todas las configuraciones válidas.

Cada configuración puede representarse como una lista de cadenas usando `K` para una posición ocupada y `.` para una posición vacía. (Indique la complejidad temporal y espacial de su algoritmo).

Ejemplo:

$$n = 3$$

Salida:

$$\begin{matrix} K & K & K \\ . & . & . \\ . & . & . \end{matrix}$$

Ejercicio Extra: Generate Parentheses (10 pts)

Dado un entero positivo `n`, genere todas las combinaciones de `n` pares de paréntesis correctamente balanceados. (Indique la complejidad temporal y espacial de su algoritmo).

Ejemplo:

$$n = 3$$

Salida:

```
["((()))", "(()())", "(())()", "()(())", "()()()"]
```